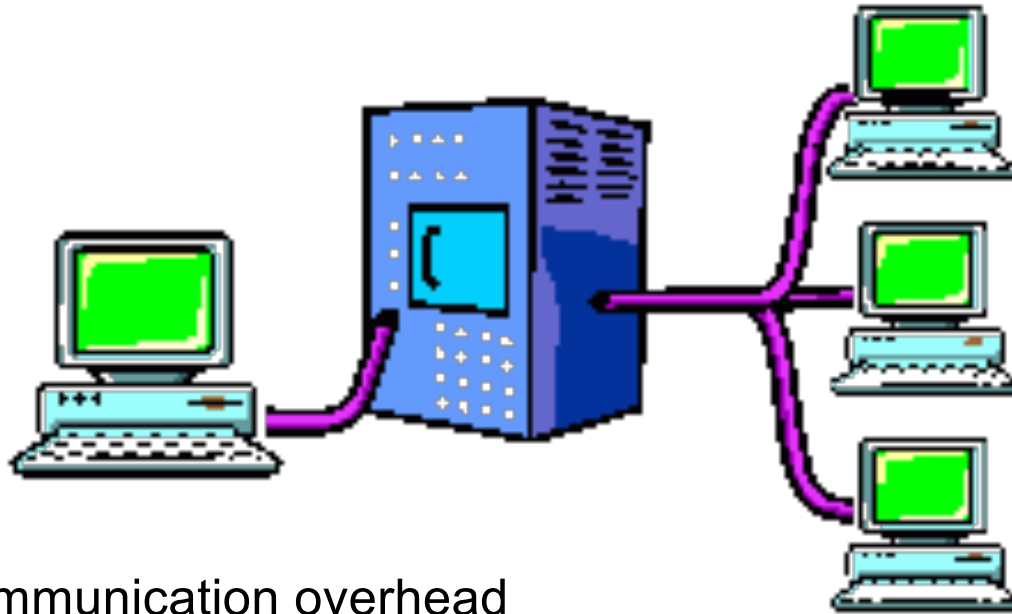


Distributed Application Architectures

this slide intentionally left blank

Monolithic Systems

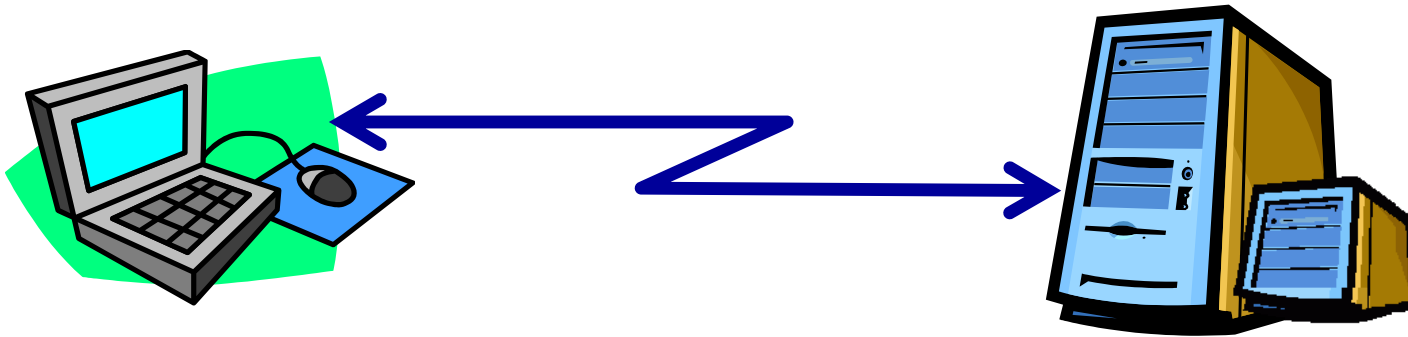
- One program or system does everything
 - exemplified by mainframe systems



- no communication overhead
- maximal coupling of components
- throughput dependent on hardware performance
- doesn't scale well

Client/Server (2-Tier)

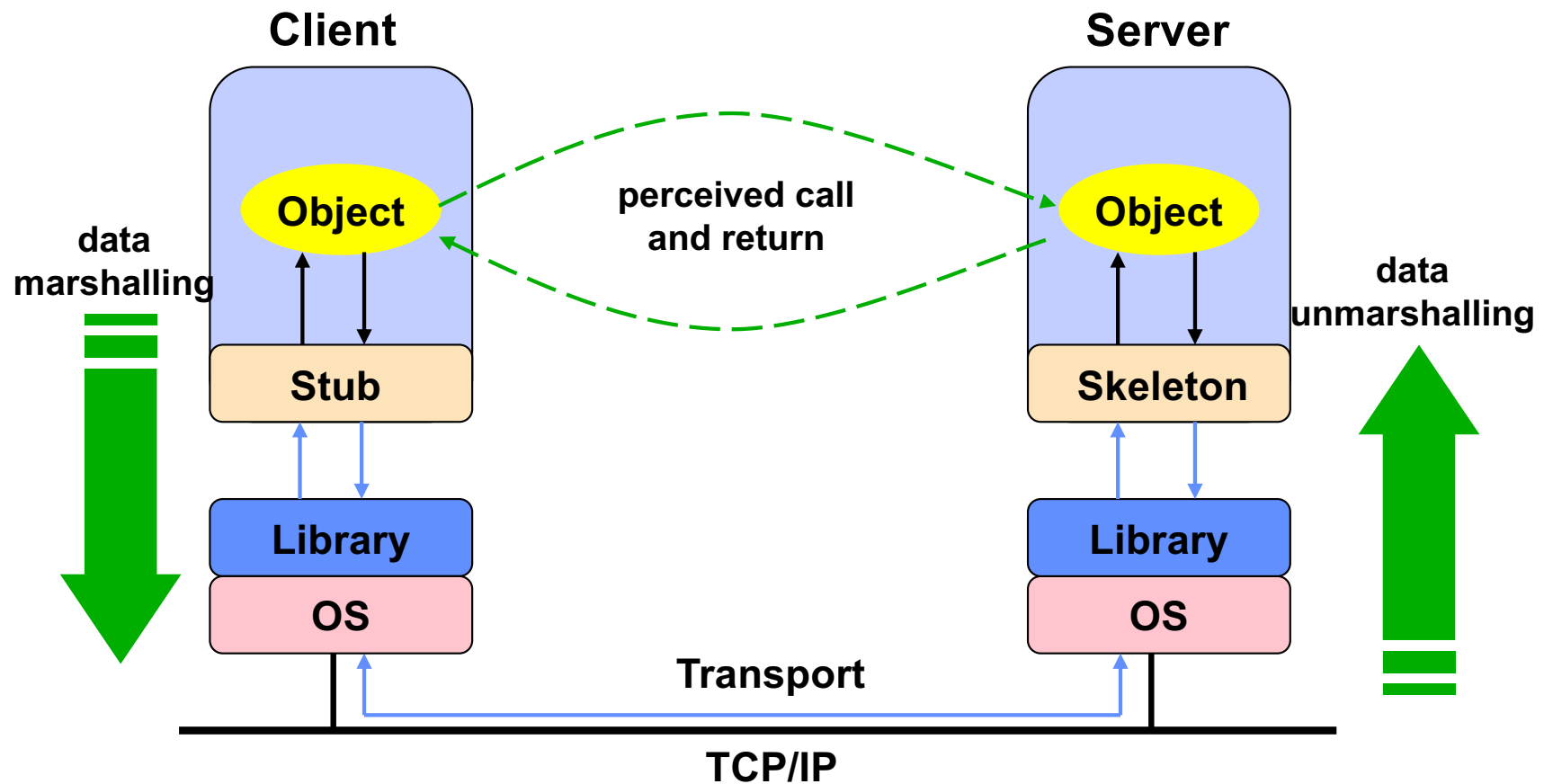
- Server provides back end services
 - typically PC client to Unix/Linux server



- business logic distributed between client and server
- server often no more than a database server
- small communication overhead
- server performance is system bottleneck
- expensive administrative overheads if fat client needs updating

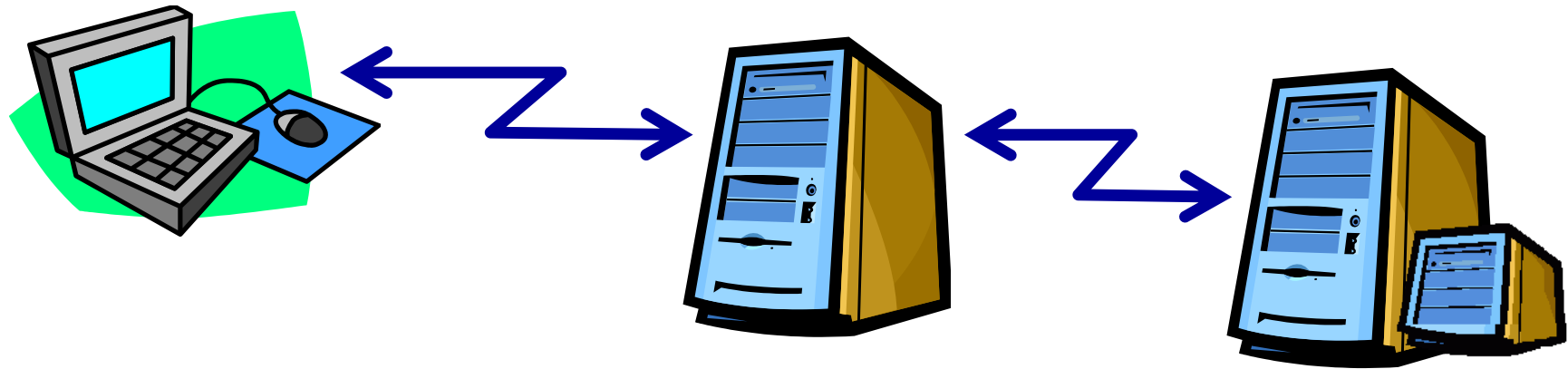
Implementing Distributed Systems

- Remote Procedure Call
 - hides network complexity behind function call interface



3-Tier Architecture

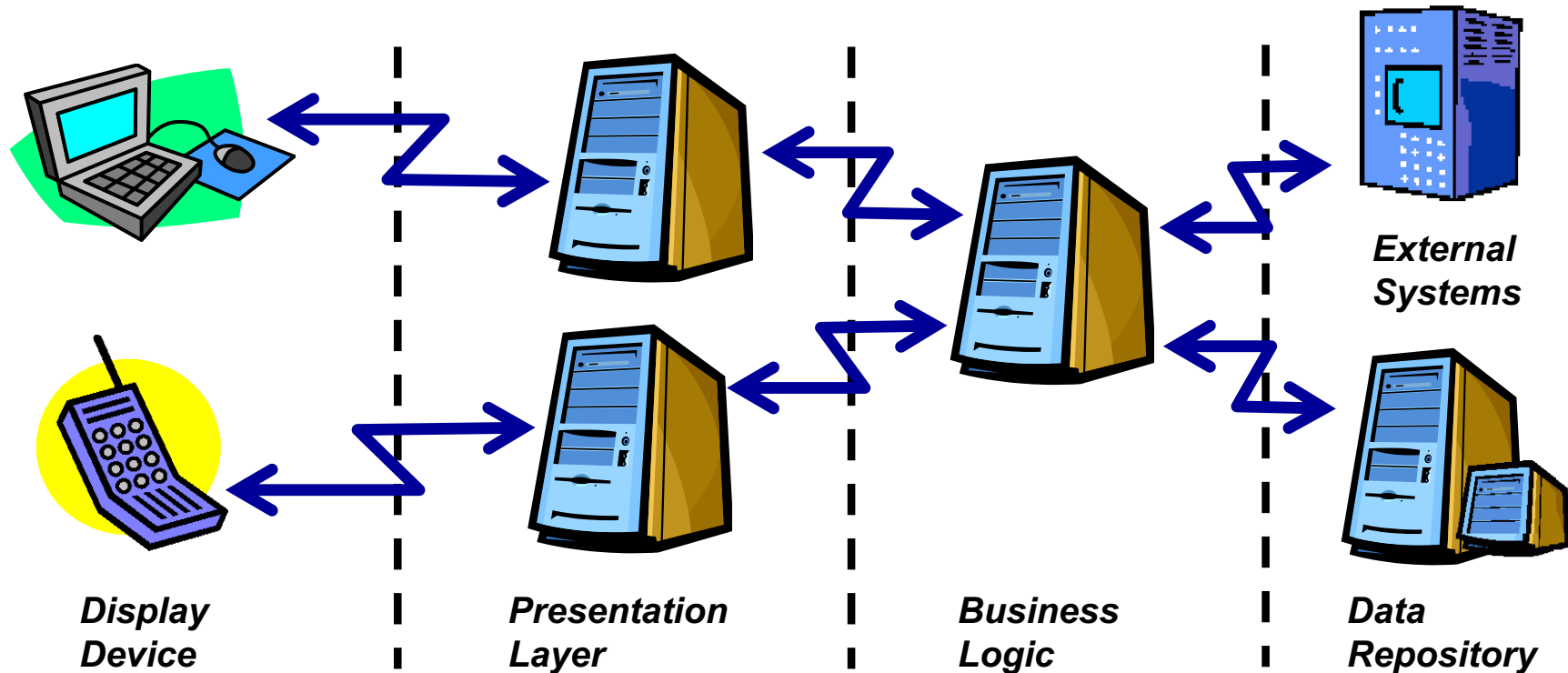
- Middleware server provides business logic
 - typically web browser client, Unix/Windows middleware



- separate backend server provides database access
- thin client is simply a presentation device
- large communication overhead
- scalable by adding additional middleware servers
- database performance is system bottleneck

N-Tier Architecture

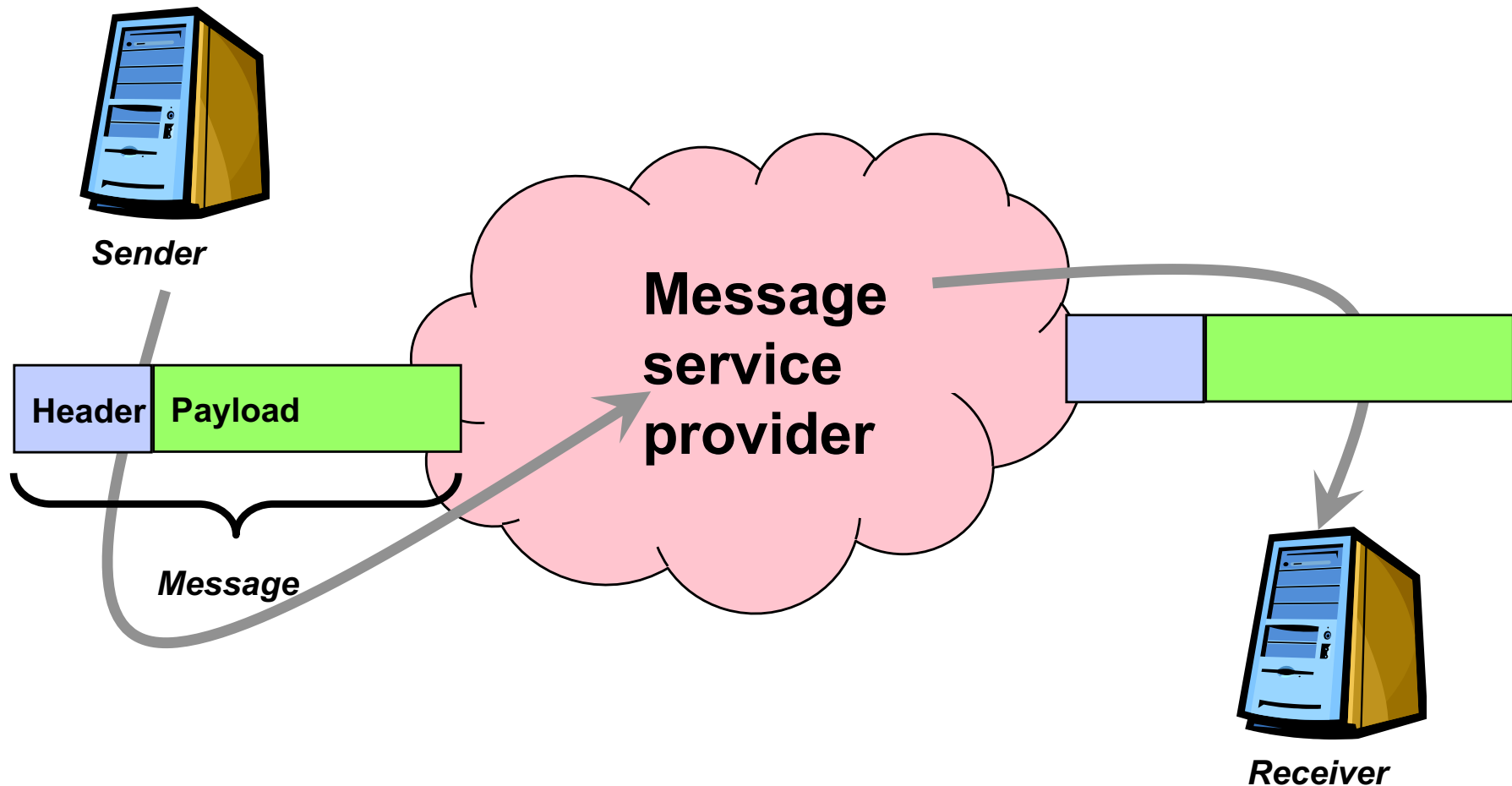
- Many specialised tiers
 - J2EE and .NET Distributed Applications



- very large communication overhead
- backend network performance is often the bottleneck
- different presentation servers for client devices

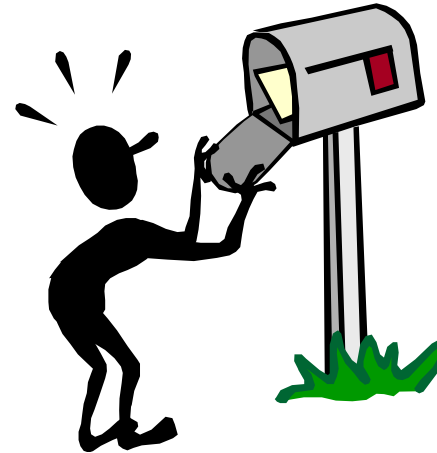
Message Oriented Middleware

- Message passing systems
 - support loosely coupled, asynchronous communication
 - can cope with high data volumes



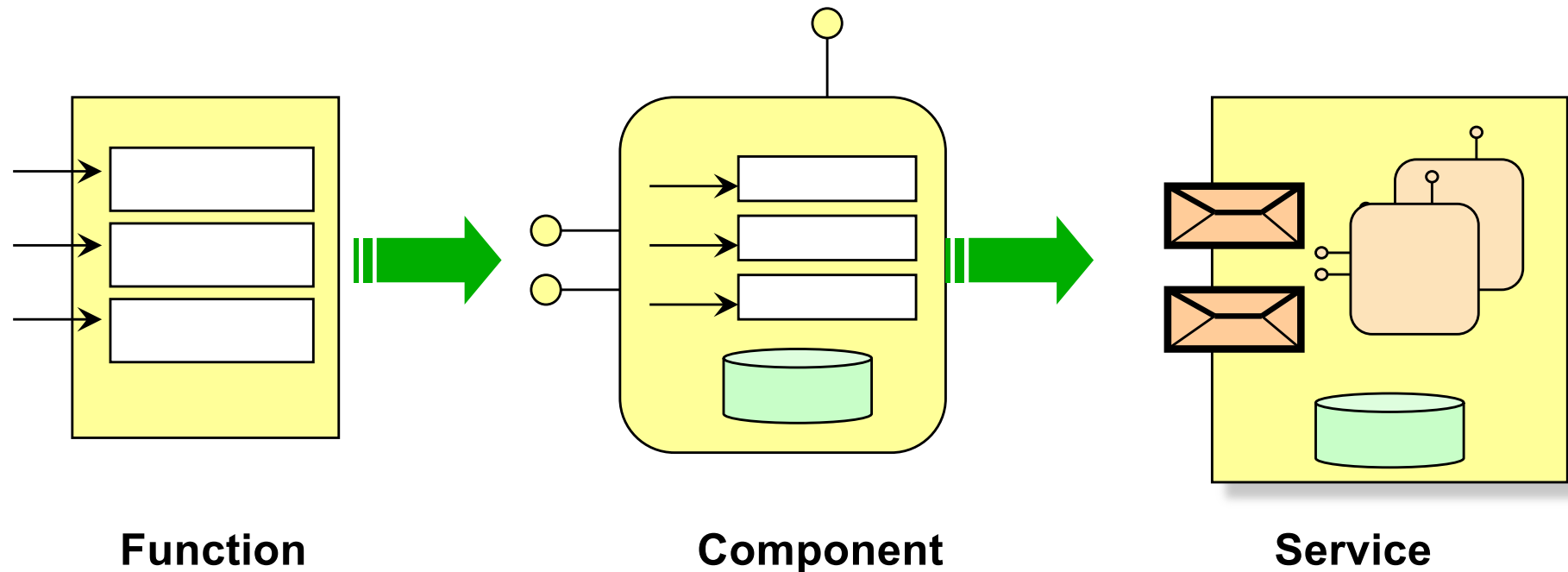
MOM Features

- Guaranteed delivery
 - does not require receiver to be active when message is sent
 - message may be persisted
- Delivery order
 - once only
 - in order
- Delivery notification
 - if required
- Dynamic routing of messages
 - receivers may migrate to other systems
- Support for transactions



Software Architecture Evolution

- Functions -> Components -> Services



- Service may contain components, private data, etc
 - messages are only means of communication

Service Oriented Architecture

- W3C defines SOA as

"a set of components which can be invoked and whose interface definitions can be published and discovered"

- Most common incarnation as Web Services

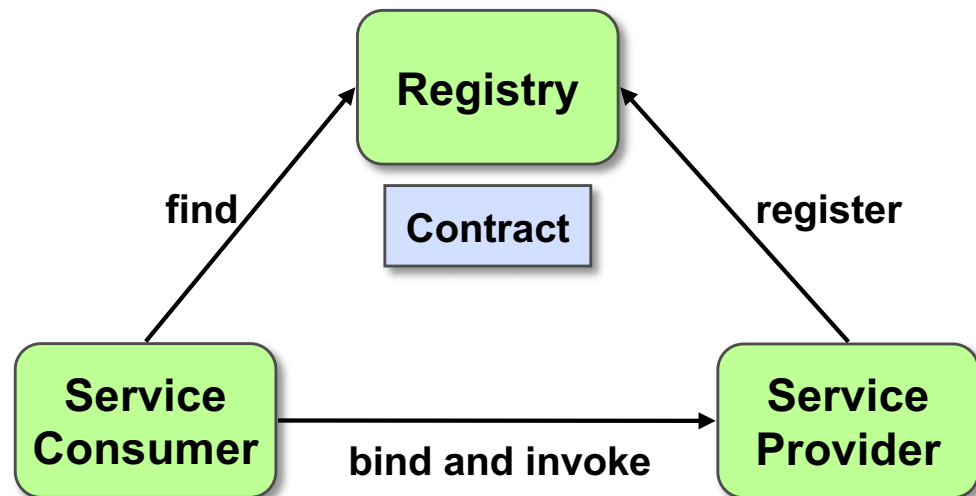
- supported by all major manufacturers
- not limited to web services

- Based on standard protocols

- XML underlying grammar for all messages
- TCP/IP basic transport protocol
- HTTP web service transport protocol
- SOAP defines message formats
- WSDL defines web service declarations

SOA Principles

- Services...
 - act as black boxes
 - define a contract between provider and consumer
 - are abstract and loosely coupled
 - may be technology-neutral
 - are autonomous
 - may be stateless
 - are composable
 - can be discovered
 - can be invoked



SOA Example

- Applications combine services
 - services are platform and vendor neutral

